

ASPECTOS TEÓRICOS DA COMPUTAÇÃO · PROJETO AV3

NeyScript

Construção de um Compilador Front-end

Análise Léxica · Sintática · Semântica e Integração

JFlex 1.9.1

CUP 11b

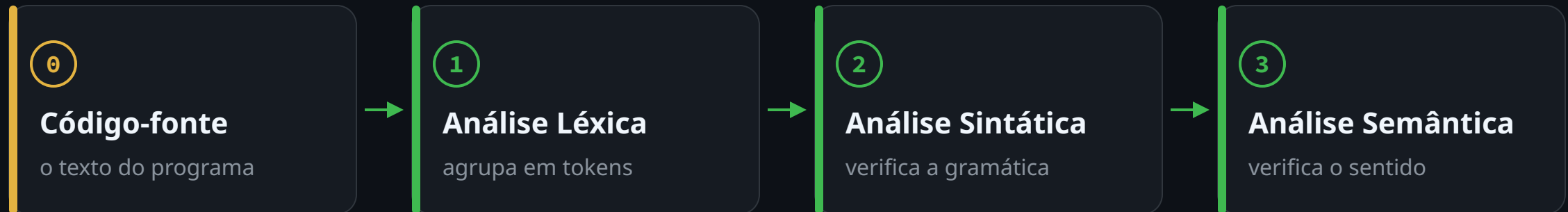
Java (JDK 11+)



Uma linguagem de programação temática (Neymar Jr.) com estrutura convencional: tipos, funções, condicionais, laços e expressões.

O que é um compilador front-end?

A parte que lê e entende o código-fonte, organizada em três fases encadeadas.



- Front-end = ler e entender o código (não gera código de máquina nesta etapa).
- Implementado com JFlex (fase léxica) e CUP (fases sintática e semântica).

JFlex gera o Scanner; CUP gera o Parser

Especificações declarativas que as ferramentas convertem em código Java.

● **src/lexer.flex**

entrada

▼ JFlex

gera

Lexer.java

Scanner (léxico)

● **src/parser.cup**

entrada

▼ CUP

gera

parser.java + sym.java

Parser (sintático/semântico)

● **src/Main.java**

entrada

▼ javac

gera

Main.class

Integração / pipeline

Do texto aos tokens

O Scanner lê caractere a caractere e produz a menor unidade com significado: o token.

- Padrões descritos com expressões regulares.
- Palavras-chave reconhecidas antes dos identificadores.
- Comentários e espaços em branco são descartados.
- %line %column rastreiam a posição de cada token.

Vocabulário → equivalente convencional

neymar	→	main
camisa10	→	int
ousadia	→	float
torcida	→	string
drible / carrinho	→	if / else
prorrogaçao	→	while
gol	→	return
assistencia	→	chamada de função

lexer.flex — padrões e palavras-chave

src/lexer.flex

```
1 Digit      = [0-9]
2 Identifier  = {Letter}({Letter}|{Digit})*
3 IntegerLit  = {Digit}+
4 FloatLit    = {Digit}+ "." {Digit}+
5
6 "camisa10"  { return symbol(sym.CAMISA10); } /* int */
7 "drible"    { return symbol(sym.DRIBLE); }   /* if */
8 "gol"       { return symbol(sym.GOL); }      /* return */
9 {Identifier} { return symbol(sym.ID, yytext()); }
```

Requisitos cobertos:

keywords

identificadores

constantes

operadores

delimitadores

comentários

A gramática da linguagem — LALR(1)

src/parser.cup

```
1 terminal String ID;
2 terminal Integer NUM_INT, NUM_FLOAT;
3
4 decl_func ::= PARTIDA tipo ID
5           LPAREN lista_params RPAREN bloco ;
6
7 stmt ::= stmt_drible      // drible / carrinho = if / else
8       | stmt_prorroga ;   // prorrogacao      = while
```

0 conflitos

62 produções

119 estados

terminais = tokens

Tabela de símbolos e checagem de sentido

Sintaxe correta não garante sentido — a semântica roda nas ações da gramática.

NOME	TIPO	CATEGORIA	PARÂMETROS
somarGols	camisa10	partida	[camisa10, camisa10]
mediaGols	ousadia	partida	[camisa10, camisa10]
anunciar	semGol	partida	[torcida]
neymar	camisa10	partida	[]

Checagens realizadas

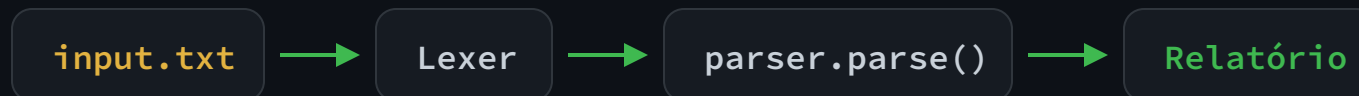
- ✓ Declaração antes do uso
- ✓ Redecaração no mesmo escopo
- ✓ Tipos compatíveis (coerção int → float)
- ✓ Aridade e tipo dos argumentos
- ✓ Retorno compatível com a função
- ✓ Condições de if / while numéricas

O driver liga o Scanner ao Parser

Uma chamada (p.parse()) dispara todo o pipeline.

src/Main.java

```
1 Lexer lex = new Lexer(new StringReader(fonte)); // 1. Scanner (JFlex)
2 parser p = new parser(lex);                    // 2. liga ao Parser (CUP)
3 p.parse();                                     // 3. token > sintaxe > semantica
4 p.tabela.imprimir();                           // 4. tabela de simbolos
```



O Parser puxa next_token() do Lexer conforme precisa (fluxo pull).

Compilação do projeto (build)

Saída real de ./build.sh — JFlex gera o Scanner, CUP gera o Parser, javac compila.

Terminal

```
$ ./build.sh
[1/3] JFlex ..... gerando Lexer.java
[2/3] CUP ..... gerando parser.java + sym.java
    0 errors and 0 warnings
    35 terminals, 25 non-terminals, 62 productions
    producing 119 unique parse states.
    0 conflicts detected (0 expected).
[3/3] javac ..... compilando classes
OK - build concluído em build/
```

Build concluído sem erros — gramática LALR(1) sem ambiguidade.

0 erros · 0 conflitos

Programa válido → aceito

Terminal

```
$ ./run.sh
```

```
ETAPA 1 - ANALISE LEXICA (Scanner JFlex)
```

```
14 1 PARTIDA
```

```
14 18 ID somarGols
```

```
41 14 NUM_INT 118
```

```
total de tokens: 237
```

```
ETAPA 2 - ANALISE SINTATICA + SEMANTICA (CUP)
```

```
>> programa analisado com sucesso.
```

```
tabela: somarGols, mediaGols, anunciar, neymar
```

```
ETAPA 3 - RELATORIO FINAL
```

GOOOOOL!! programa aceito sem erros léxicos, sintáticos ou semânticos.

237 tokens · 0 erros

Programa com erros → diagnóstico completo

Terminal

```
$ ./run.sh input_erros.txt
ERRO SEMANTICO (linha 9): partida 'dobro' ja declarada
ERRO SEMANTICO (linha 22): 'y' (camisa10) = ousadia
ERRO SEMANTICO (linha 31): 'dobro' espera 1 argumento, recebeu 3
ERRO SEMANTICO (linha 34): arg 1: esperado camisa10, recebeu torcida
ERRO SEMANTICO (linha 36): partida 'inexistente' nao declarada
... e mais 6 erros, cada um com a sua linha
ETAPA 3 - RELATORIO FINAL
```

CARTÃO VERMELHO — 11 erro(s) encontrado(s)

não para no 1º erro

Resumo dos testes executados

Cada caso foi rodado e conferido contra o resultado esperado.

CASO DE TESTE	ENTRADA	RESULTADO OBTIDO
Compilação	<code>./build.sh</code>	✓ 0 erros · 0 conflitos · classes geradas
Programa válido	<code>input.txt</code>	✓ 237 tokens · 0 erros · GOL
Programa com erros	<code>input_erros.txt</code>	✓ 11 erros detectados · CARTÃO VERMELHO
Análise léxica isolada	<code>./run.sh --lex</code>	✓ 237 tokens listados

Todos os casos retornaram o resultado esperado — o front-end opera de ponta a ponta.

Requisitos do trabalho — todos atendidos

Léxico (JFlex)

- ✓ keywords
- ✓ identificadores
- ✓ constantes int / float
- ✓ operadores
- ✓ delimitadores
- ✓ comentários // e /* */

Sintático + Semântico (CUP)

- ✓ estrutura do programa
- ✓ declaração de variáveis
- ✓ controle if-else e while
- ✓ definição e chamada de funções
- ✓ expressões e atribuições
- ✓ checagens semânticas

Integração

- ✓ Scanner + Parser operando juntos
- ✓ input.txt com tudo
- ✓ demonstração ponta a ponta
- ✓ saída esperada (GOL / erros)

09 · ENCERRAMENTO

Obrigado!

Léxico (JFlex) + Sintático e Semântico (CUP) + Integração (Java).

- ✓ Programa correto → aceito com a tabela de símbolos.
- ✓ Programa com erro → 11 diagnósticos, cada um com a sua linha.

github.com/joaoviitorsx/neyscript

